GOLD STANDARD

PDF REPORT

OCTOBER 31, 2026

GOLD STANDARD

TEAM INFORMATION



Senzo
u24691497



Njabulo
u24981712



Liam
u24685993



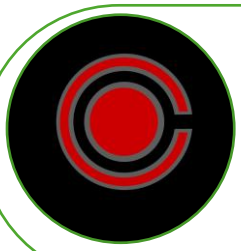
Livhuwani
u25449029



Navela
u25254686



Lwandiso
u25405064



Tebello
u25256387

TEAM CONTRIBUTIONS

Njabulo u24981712:

1. **Code - TUI, Chain of Responsibility**
2. **Insurance Activity Diagram**
3. **Sequence Diagram - Responding to a Task (with Liam)**
4. **UML Class Diagram(Most Parts)**

George u25405064:

1. **Docker**
2. **Doxygen**
3. **Memento and strategy (implementation),**
4. **State diagram and the ProductionWorkflow scenario**
5. **Manufacturing Activity Diagram**

Tebello u25256387:

1. **Façade Design Pattern**
2. **Participants**
3. **Builder Patten**
4. **Unit testing**

Livhuwani u25449029

1. **State pattern implementation**
2. **Doxygen comments**
3. **Sequence diagram(approval and escalation)**
4. **UML Class Diagram(Partially)**

Liam u24685993:

1. **Composite Implementation**
2. **Observer Implementation**
3. **Concrete product implementation: HumanTask and AutomatedTask**
4. **Responding to a Task sequence diagram (with Njabulo)**
5. **Checked the specification, identified missing functionality and added missing overdue-task support. (small contribution)**

Senzo u24691497:

1. **Research**
2. **Decorator and factory method implementation)**
3. **TIVIDY sequence diagram (with Tebello)**
4. **GitHub Action CI/CD pipeline**
5. **Presentation slides**

Navela u25254686:

1. **Iterator Implementation**

2. **Generic Workflow Activity Diagram(Workflow Instance Creation until Completion)**
3. **Final Report**
4. **UML Class Diagram(Partially)**

RESEARCH WORKFLOW

Workflow and Workflow management system definition.

A workflow is the way that a particular type of work is organized, or the order of the stages in a particular work process.A Workflow Management System is a system that defines, manages, and executes workflows using software based on the workflow logic. It automates the executions of applications and is robust against performance and failures.

Difference between a workflow definition and a running workflow instance .

A workflow definition is the static blueprint or template that outlines the rules, steps, and logic of a process, while a running workflow instance is the actual active execution of that blueprint for a specific task or event.

Representation of activities or work items.

In workflow systems, activities are usually modeled as named process steps with control-flow links, data inputs and outputs, resource bindings, and sometimes application calls, while work items are the runtime units offered to a person, device, or service for one specific case.

Way in which work items are assigned and routed.

Work items are assigned and routed based on rules defined in the workflow definition, which are then executed for specific users or systems within the running workflow instance.

How approvals, rejection and escalation are handled.

Approvals, rejections, and escalations are handled in a running workflow instance by executing conditional gating and timing rules predefined in the workflow definition.

How participants, roles and responsibilities are represented .

Participants are modelled as resources, meaning the humans or systems that do work, and roles group those resources so that work can be offered to a role instead of a named person. Responsibility attaches when a work item is allocated to a specific resource, who is then accountable for it and can pass it to a delegate (Russell et al. , 2004).

How dependencies, events and failures are handled.

Workflow systems typically treat dependencies as explicit execution constraints, events as signals that change process state or trigger actions, and failures as exceptions that require rollback, compensation, alternative routing, or recovery workflows.

How workflow systems interact with external systems.

Workflow systems interact with external systems through standard interfaces, and the engine itself stays separate from them. The WfMC Reference Model says the engine interacts with the external resources needed to carry out activities (WfMC, 1993), and it defines interfaces for process definition tools, for interoperability between engines, for end-user client tools and for administration and monitoring tools (WfMC, 1996).

How workflow history or audit information is recorded.

Workflow history is recorded by the workflow engine, which writes an audit record every time something happens. Each record names the event and has a process instance ID, that links all the records of one case, and an activity instance ID that is unique to that case. Those records are kept together with process and role data, so the history of who did what can be checked against the process definition.

How running workflows are monitored and reported on.

Running workflows are monitored by capturing process state, task status, timestamps, and execution events as work progresses, then exposing that information through a central layer for users and tools. Reporting usually sits on top of that monitoring layer as real-time dashboards, alerts, and exportable summaries tailored to operational and managerial decisions.

Reasoning on why workflow engines are reusable across different processes.

It is reusable because the process logic is kept out of the engine and stored in a process definition that the engine reads and runs. The engine only provides the generic parts and everything else that differs between processes is configured in the definition. Therefore, the same engine runs a different process just by loading a different definition.

What information belongs to a workflow definition and to a running instance?

The definition holds the reusable template: the activities, procedure rules, control data, sub-process references, per-activity settings and transition rules. The running instance holds what belongs to one case: its own data and variables, case's current state and of each activity, the task instances with who took them, and the audit records of what happened.

How real nested activities or sub-processes are modeled in real workflow systems.

They are modeled as a separately defined process that a parent definition can refer to, so it makes up part of the overall process. A sub-process therefore behaves like a single activity from the parent's point of view, while being a full process underneath. Links to parent via the root process instance ID.

Decisions that are normally configurable rather than hard-coded into the workflow engine.

The decisions that vary between organisations are kept in the process definition or in configuration, not in the engine's code. These are activities and the order they run in, the transition conditions and flow rules, and the start and termination conditions of a process.

How are changes to workflow definitions handled while work is already in progress.

Changes to workflow definitions while work is already in progress are typically handled through parallel versioning, where existing running instances are safely isolated and left to finish using their original blueprint version. Simultaneously, the engine routes all incoming new traffic to the modified definition.

REFERENCES IBM (n.d.) Workflow definitions and workflow instances , IBM OpenPages 9.0.0 documentation. Available at: https://www.ibm.com/docs/en/openpages/9.0.0?topic=fundamentals-workflow-definitions-wor kflow-instances (Accessed: 3 October 2026).

Van der Aalst, W.M.P., ter Hofstede, A.H.M., Kiepuszewski, B. and Barros, A.P. (2003) 'Workflow patterns', Distributed and Parallel Databases , 14(1), pp. 5-51. doi: 10.1023/A:1022883727209.

Ismail Sayyad (2026) Power Automate Approval with Reminder & Escalation . Available at: https://www.youtube.com/watch?v=UjcBdwmVXYs (Accessed: 3 October 2026).

Camunda (n.d.a) Extension attributes , Camunda 7 manual. Available at: https://docs.camunda.org/manual/7.21/reference/bpmn20/custom-extensions/extension-attributes/ (Accessed: 3 October 2026.

Workflow Management Coalition (1993) Workflow events paper . Available at: https://wfmc.org//wp-content/uploads/2022/09/Workflow_events_paper.pdf (Accessed: 3 October 2026).

Workflow Management Coalition (1996) Workflow interoperability . Available at: https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/if4/if4-a.html (Accessed: 3 October 2026).

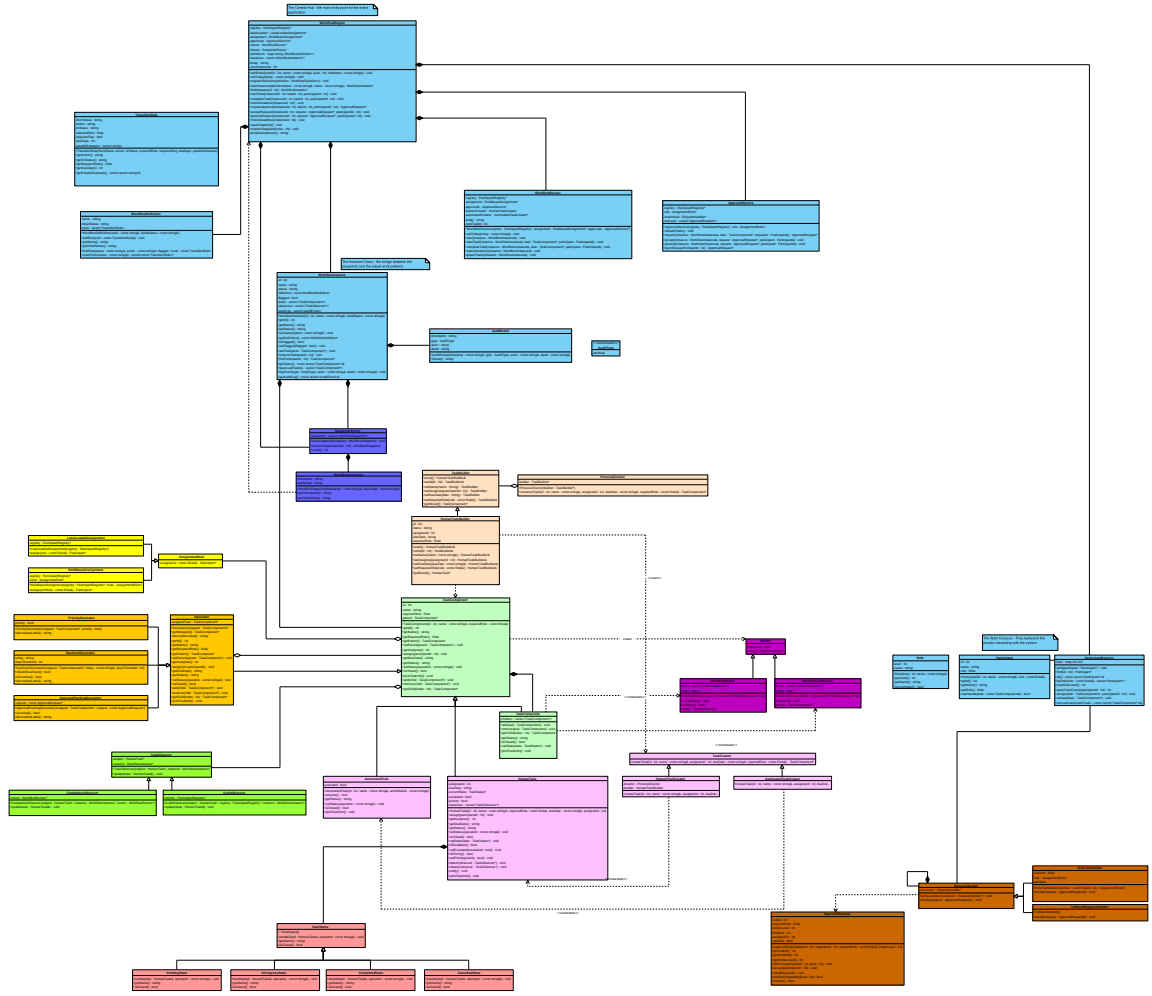
Workflow Management Coalition (1996) Workflow audit data specification . Available at: https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/if5/if59611.html (Accessed: 3 October 2026).

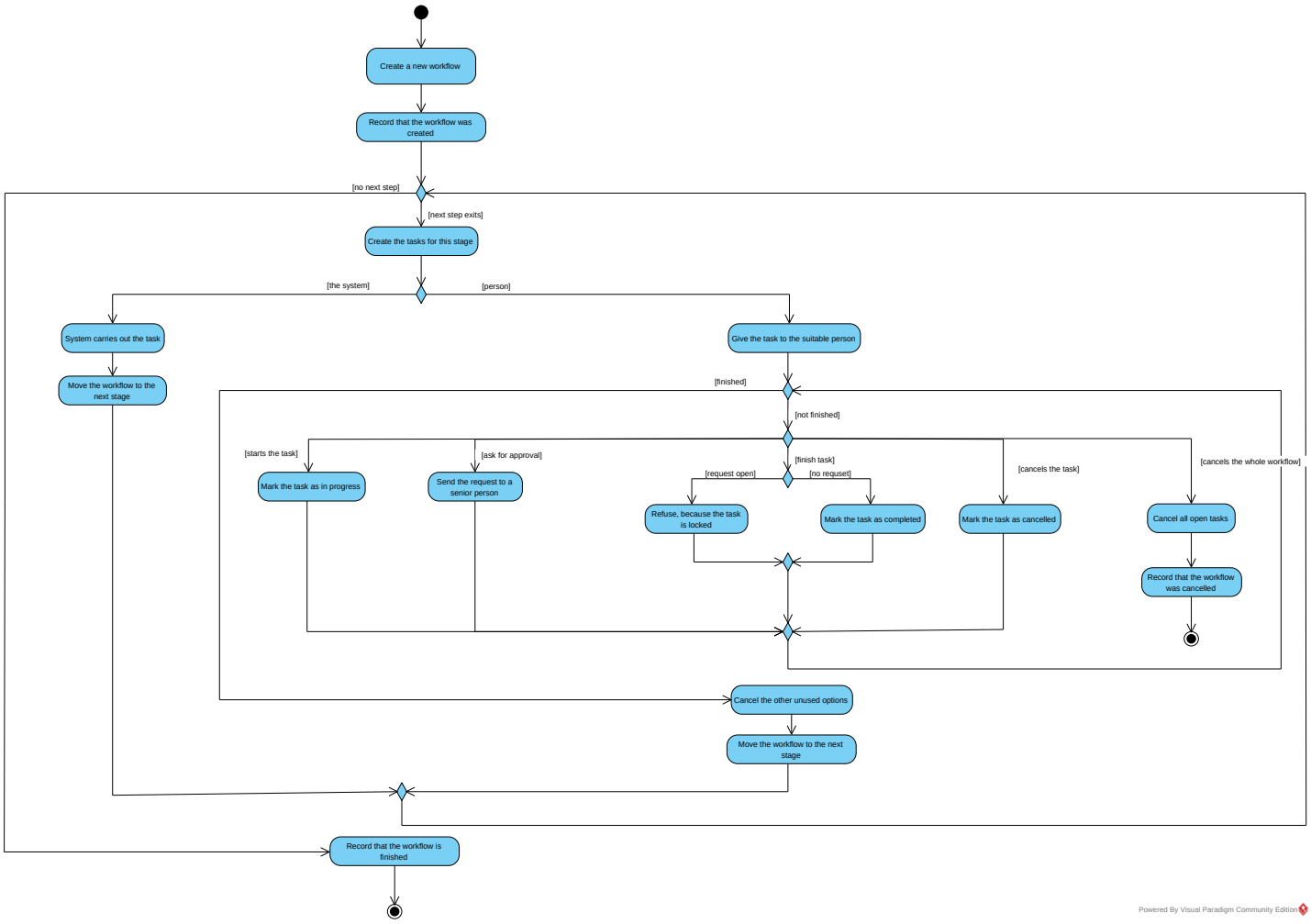
Russell, N., ter Hofstede, A.H.M., Edmond, D. and van der Aalst, W.M.P. (2004) Workflow resource patterns . Eindhoven: Eindhoven University of Technology. Available at: https://research.tue.nl/en/publications/workflow-resource-patterns/ (Accessed: 3 October 2026).

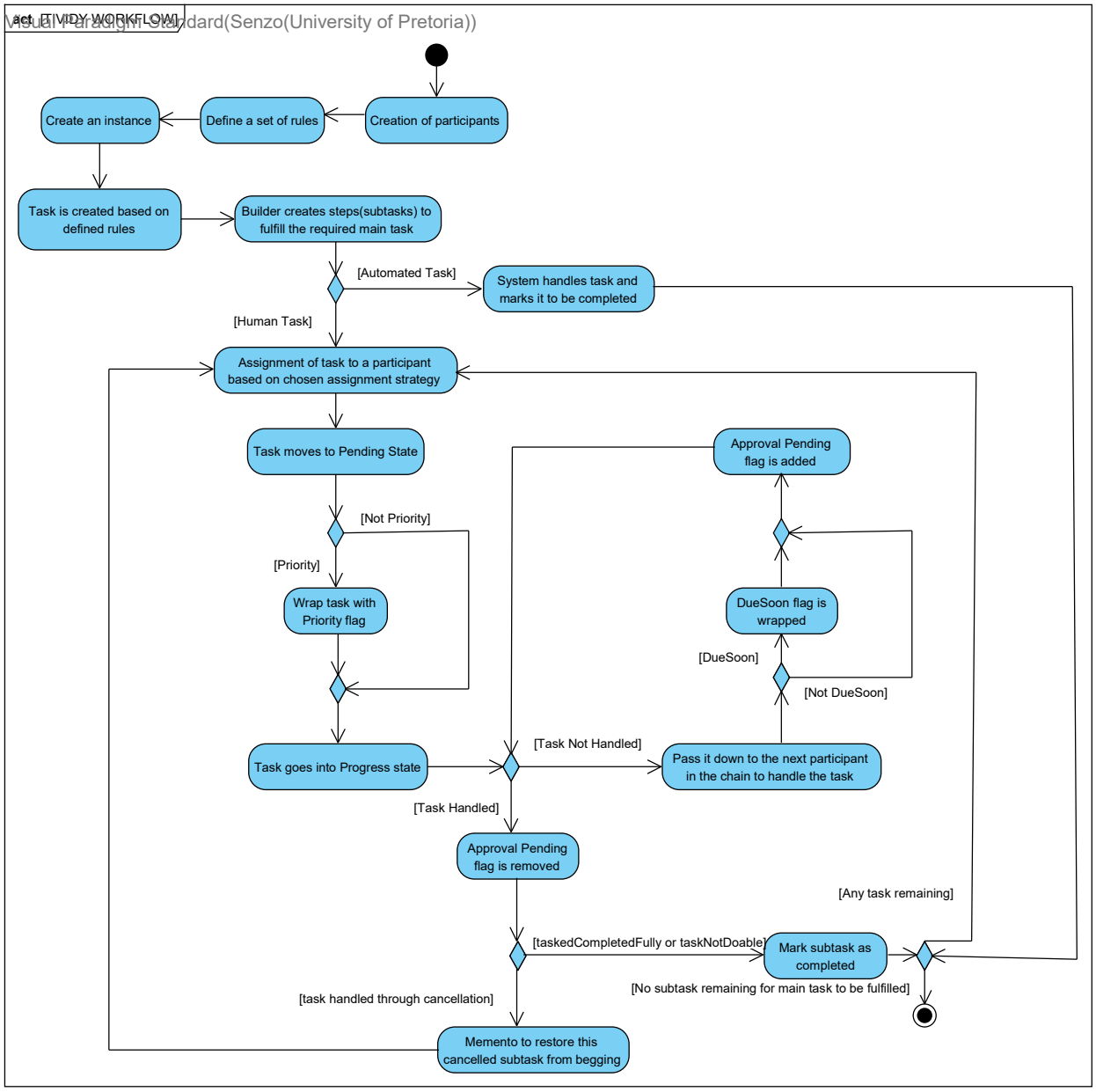
Workflow Management Coalition (1996) Terminology & glossary . Available at: https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/glossary/glossary.html (Accessed: 3 October 2026).

Camunda (2026) Process instance migration , Camunda 8 documentation. Available at: https://docs.camunda.io/docs/components/concepts/process-instance-migration/ (Accessed: 3 October 2026) 4

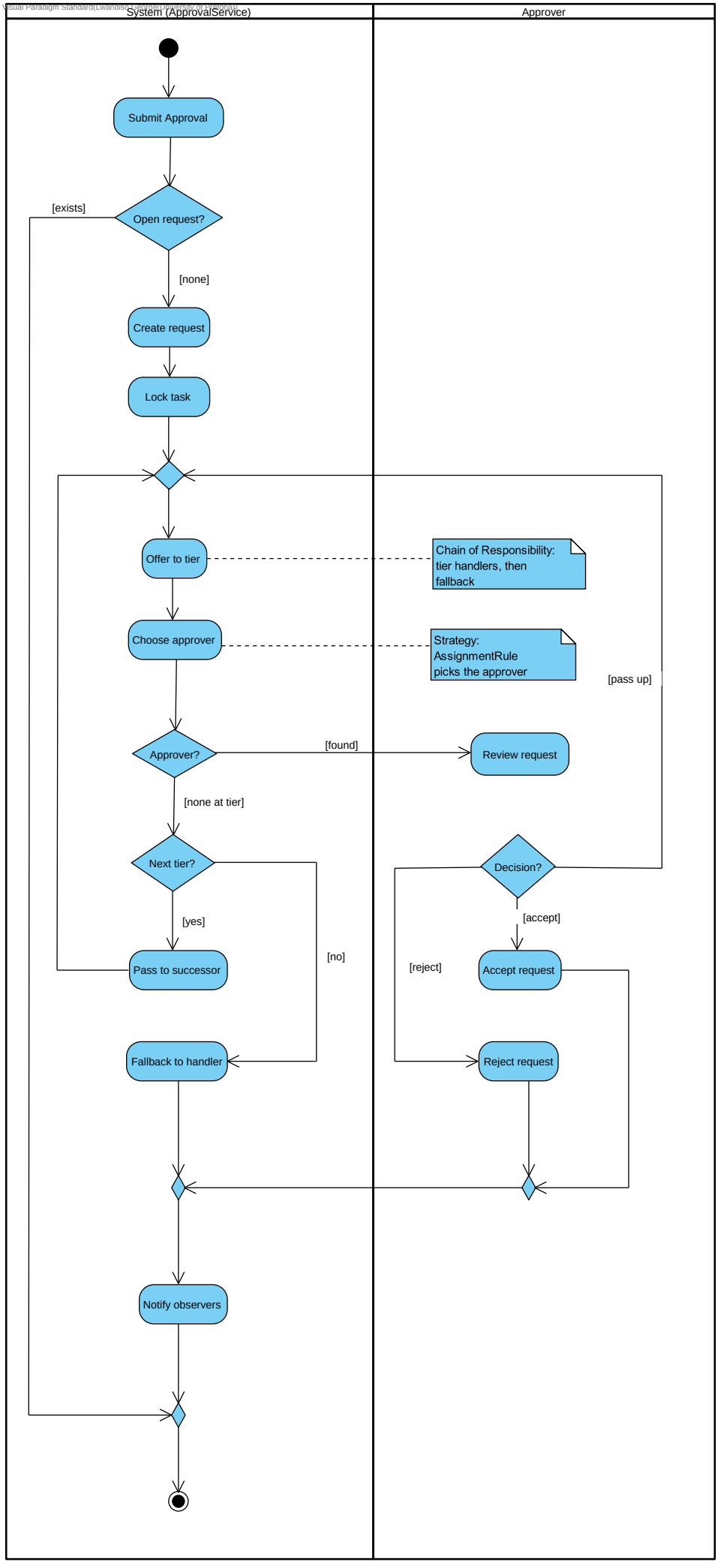
DIAGRAMS


The Central Hub - the main entry point for the entire framework
WorkflowEngine
TaskScheduler
WorkflowExecutor
ParallelWorkflowEngine
The Scheduler Clock - the bridge between the Blueprints and the Actor and Ledgers
WorkflowInstance
ProcessInstance
TaskManager
TaskBuilder
ProcessComponent
HumanTaskHandler
HumanTaskManager
ProcessDefinition
TaskComponent
Task
TaskExecutor
ProcessDefinitionBuilder
ProcessDefinitionManager
ProcessDefinitionRepository
ProcessDefinitionValidator
ProcessDefinitionSerializer
ProcessDefinitionVersionManager
ProcessDefinitionMigrationManager
ProcessDefinitionMigrationStrategy
ProcessDefinitionMigrationStrategyFactory
ProcessDefinitionMigrationStrategyRegistry
ProcessDefinitionMigrationStrategyExecutor
ProcessDefinitionMigrationStrategyExecutorFactory
ProcessDefinitionMigrationStrategyExecutorRegistry
ProcessDefinitionMigrationStrategyExecutorManager
ProcessDefinitionMigrationStrategyExecutorValidator
ProcessDefinitionMigrationStrategyExecutorSerializer
ProcessDefinitionMigrationStrategyExecutorDeserializer

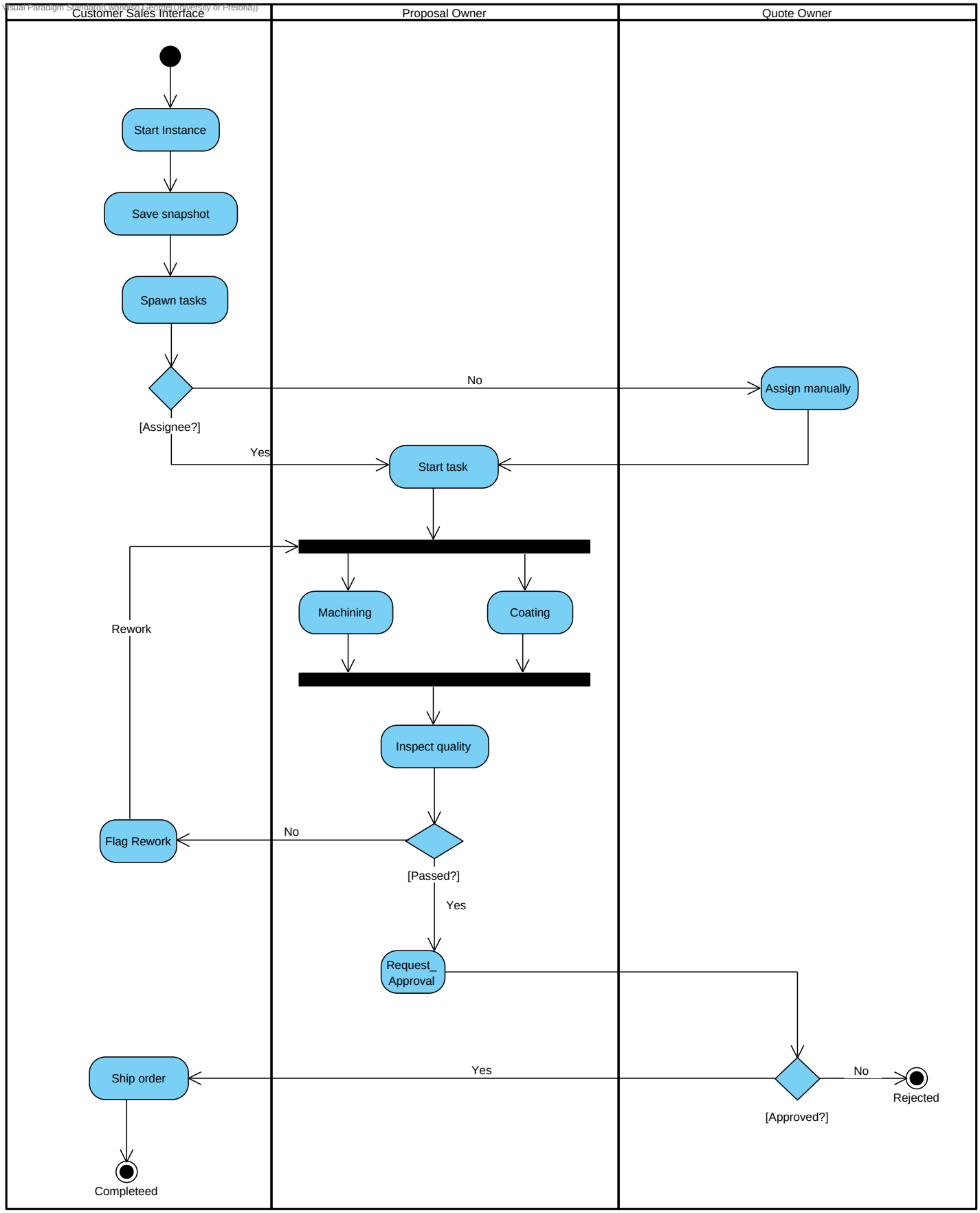

Create a new workflow
Record that the workflow was created
[no next step]
[next step exits]
Create the tasks for this stage
[the system]
[person]
System carries out the task
Move the workflow to the next stage
Give the task to the suitable person
[finished]
[not finished]
[starts the task]
[ask for approval]
[finish task]
[request open]
[no requset]
[cancels the task]
[cancels the whole workflow]
Mark the task as in progress
Send the request to a senior person
Refuse, because the task is locked
Mark the task as completed
Mark the task as cancelled
Cancel all open tasks
Record that the workflow was cancelled
Cancel the other unused options
Move the workflow to the next stage
Record that the workflow is finished
Powered By Visual Paradigm Community Edition

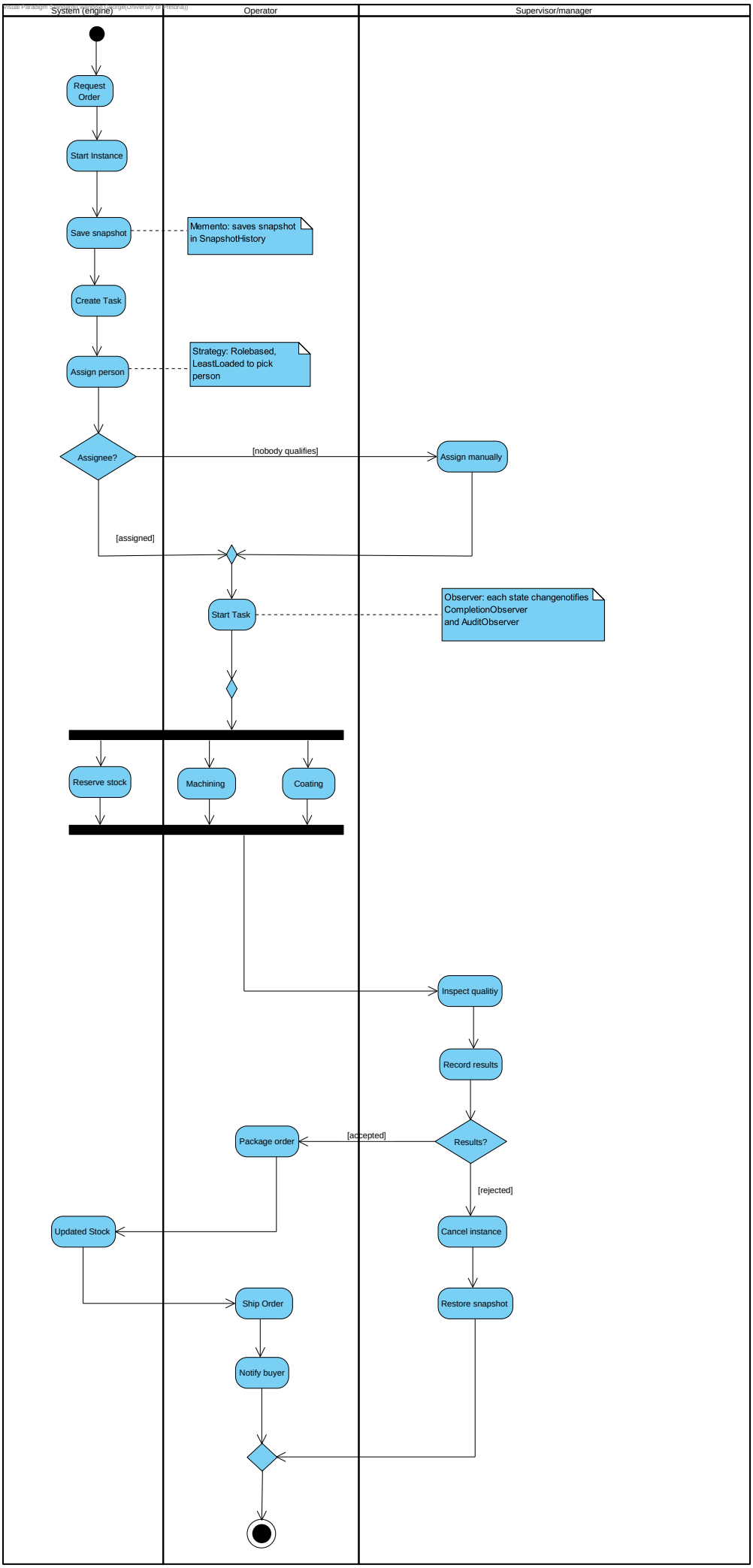

act [TIDY WORKFLOW]
Visual Paradigm Standard(Senzo(University of Pretoria))
Creation of participants
Define a set of rules
Create an instance
Task is created based on defined rules
Builder creates steps(subtasks) to fulfill the required main task
[Automated Task]
System handles task and marks it to be completed
[Human Task]
Assignment of task to a participant based on chosen assignment strategy
Task moves to Pending State
[Not Priority]
[Priority]
Wrap task with Priority flag
Task goes into Progress state
[Task Not Handled]
Pass it down to the next participant in the chain to handle the task
Approval Pending flag is added
DueSoon flag is wrapped
[DueSoon]
[Not DueSoon]
[Task Handled]
Approval Pending flag is removed
[Any task remaining]
[taskedCompletedFully or taskNotDoable]
Mark subtask as completed
[No subtask remaining for main task to be fulfilled]
[task handled through cancellation]
Memento to restore this cancelled subtask from begging

Visual Paradigm Standard(Lwandile Kwanda(Individual Academic))

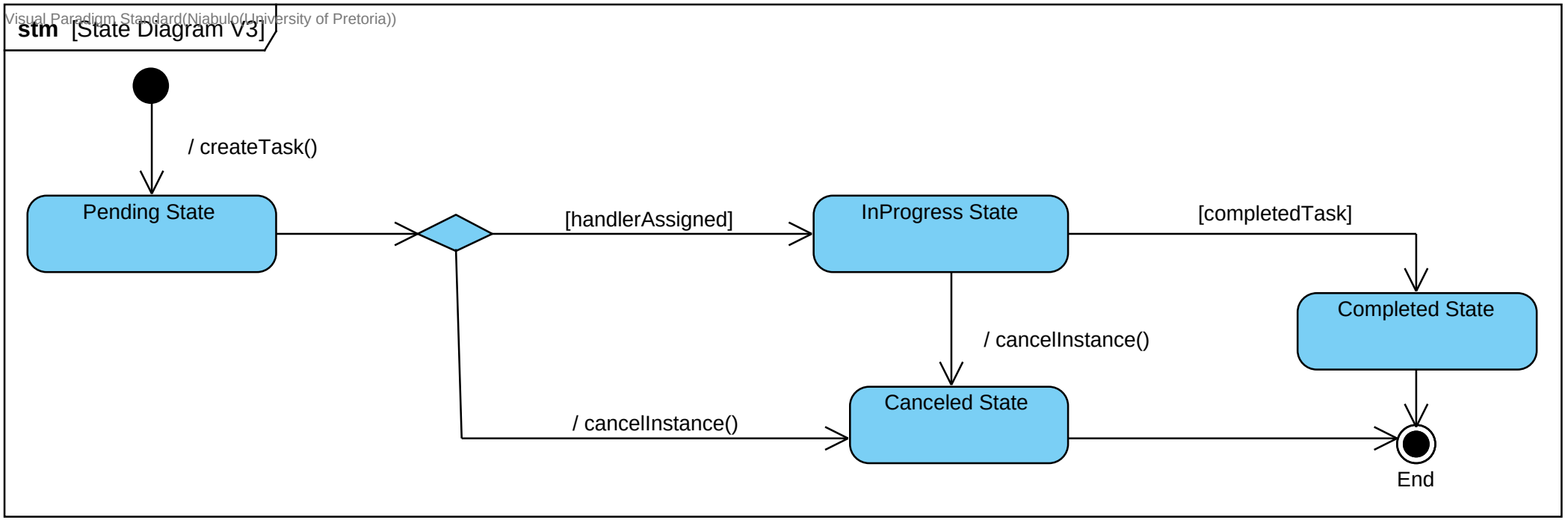


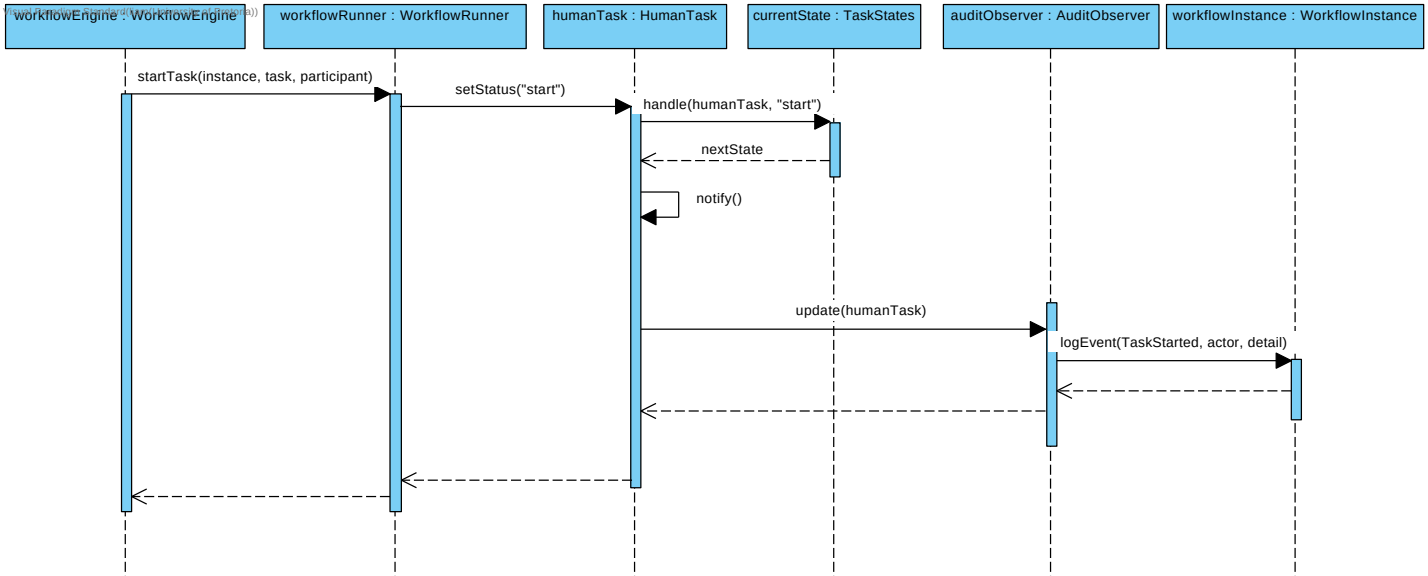
Visual Paradigm Standard(Lwandisa Leonard(University of Pretoria))



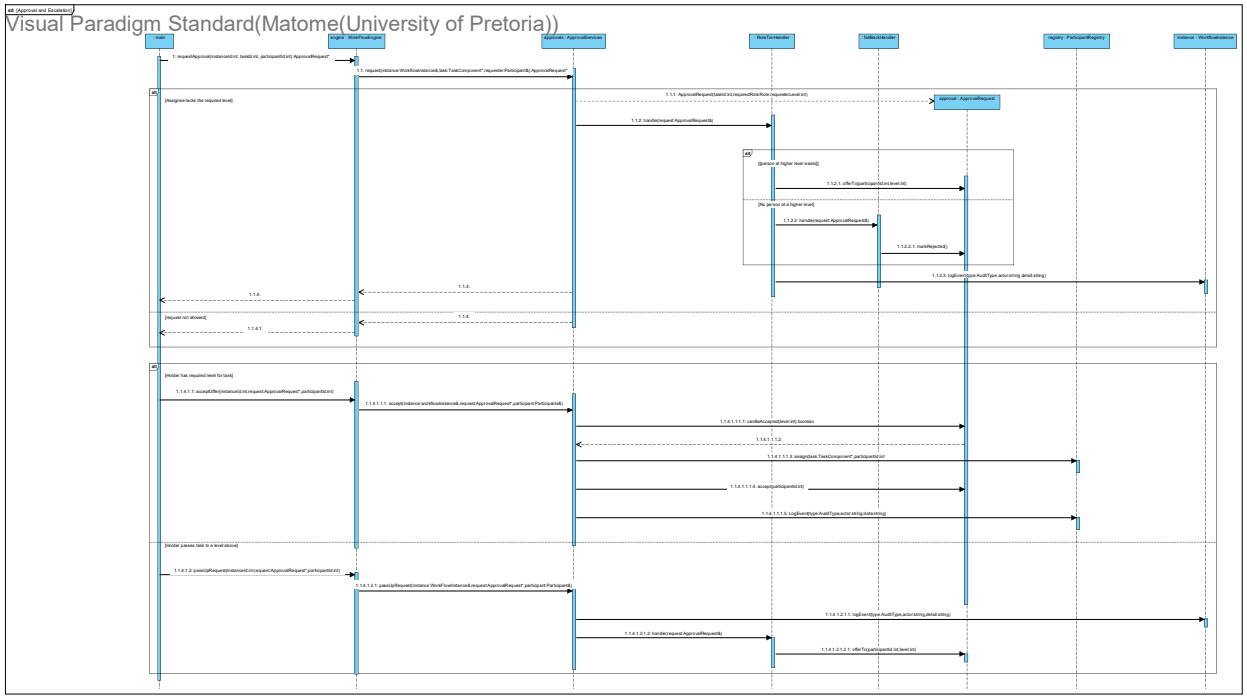

System (engine)
Operator
Supervisor/manager
Request Order
Start Instance
Save snapshot
Memento: saves snapshot in SnapshotHistory
Create Task
Assign person
Strategy: Rolebased, LeastLoaded to pick person
Assignee?
[nobody qualifies]
Assign manually
[assigned]
Start Task
Observer: each state changenotifies CompletionObserver and AuditObserver
Reserve stock
Machining
Coating
Inspect qualitiy
Record results
Results?
[accepted]
Package order
[rejected]
Cancel instance
Restore snapshot
Updated Stock
Ship Order
Notify buyer

Visual Paradigm Standard(Niabulo(University of Pretoria))




workflowEngine : WorkflowEngine
workflowRunner : WorkflowRunner
humanTask : HumanTask
currentState : TaskStates
auditObserver : AuditObserver
workflowInstance : WorkflowInstance
startTask(instance, task, participant)
setStatus("start")
handle(humanTask, "start")
nextState
notify()
update(humanTask)
logEvent(TaskStarted, actor, detail)

Visual Paradigm Standard(Matome(University of Pretoria))



1. Insurance Claim Processing Scenario

Processes & Stages

- **Initial Status:** ClaimSubmitted
- **Lifecycle Stages:** ClaimSubmitted → InitialReview → AssessmentAndApproval → PayoutProcessing (or ClaimRejected)

Participants & Roles

- **Level 1 (Junior Claims Adjuster):** Handles standard data verification and basic damage estimates.
- **Level 2 (Senior Adjuster / Assessor):** Reviews medium-tier claim adjustments and complex damage reports.
- **Level 3 (Claims Manager):** Authorizes high-value payouts and exceptions requiring executive sign-off.
- **Level 0 (Automated System):** Executes automated fraud-scoring checks and database record updates.

Branching & Decision Rules

- **Guarded Rows & Flags:** If an automated fraud check flags a claim (flagged = true), the workflow definition intercepts the standard flow, taking a guarded transition rule that routes the claim directly to a Senior Adjuster instead of the junior queue.
- **Approval Escalation (Chain of Responsibility):** If an adjuster assesses damage exceeding their authority level, they request approval. The ApprovalService offers the request up the chain tier-by-tier

(RoleTierHandler) until a manager accepts it or the fallback handler rejects it.

Failure Paths & Exceptions

- **Approval Lock:** While an approval request is pending, the task is locked. Attempting to complete it throws an IllegalTransitionError.
- **Rejection / Refusal:** If an approval request reaches the top of the chain and is rejected, the task stays with the original requester, and the definition's fallback rule triggers a rejection path.
- **Cancellation:** If a claimant withdraws, the engine cancels all open tasks and logs InstanceCancelled.

Final Outcomes

- **Successful Outcome:** InstanceFinished following a completed electronic payout transfer.
- **Alternative Outcome:** Claim rejected or instance explicitly cancelled.

2. Custom Manufacturing Order Scenario

Processes & Stages

- **Initial Status:** OrderReceived
- **Lifecycle Stages:** OrderReceived → ComponentFabrication → AssemblyAndQC → ShippingAndFulfillment

Participants & Roles

- **Level 1 (Floor Technician / Machinist):** Executes physical cutting, material handling, and basic machine operation.
- **Level 2 (Lead Engineer / QC Inspector):** Oversees precision calibration, conducts quality checks, and handles manufacturing deviations.
- **Level 3 (Plant Operations Manager):** Approves costly material swaps, custom re-tooling budgets, or deadline extensions.
- **Level 0 (Automated System):** Triggers CNC machine g-code execution or inventory deduction scripts.

Branching & Decision Rules

- **Parallel Subtasks (Composite Pattern):** When entering ComponentFabrication, the transition rule spawns a composite task containing parallel subtasks: "*Cut Raw Steel*" and "*Program CNC Machine*". Both must run and complete concurrently.
- **Alternative Paths:** If a custom part fails quality inspection, a guarded transition rule routes the workflow back to a remediation stage instead of direct shipping.

Failure Paths & Exceptions

- **Subtask Stall / Failure:** If one parallel subtask is cancelled or blocked, the composite task cannot close, halting workflow advancement until resolved

(Rule 7: a composite completes only when all subtasks close).

- **Authority Deficit:** If a technician requires an expensive tooling change beyond their role level, they must trigger an approval request up to the Plant Manager.

Final Outcomes

- **Successful Outcome:** InstanceFinished once the final shipping verification automated task executes successfully.
- **Alternative Outcome:** Order cancelled due to supply chain shortages or client cancellation.

PATTERN ROLES AND JUSTIFICATIONS

Creational Patterns

1. Factory Method

- **Classes:** TaskCreator (abstract), HumanTaskCreator, AutomatedTaskCreator

- **Role:** Encapsulates the instantiation logic for creating different types of tasks without tightly coupling creation code to specific task classes.

- **Justification:** Allows the runner to spawn human or automated tasks generically based on definition rows without hardcoding instantiation logic.

2. Builder

- **Classes:** TaskBuilder (abstract), HumanTaskBuilder, ProcessDirector

- **Role:** Separates the construction of complex tasks from their representation. ProcessDirector coordinates the step-by-step building process in a standard sequence via HumanTaskBuilder.

- **Justification:** Ensures that every task (with IDs, names, roles, assignees, due dates) is built uniformly via a consistent director-builder pipeline.

Structural Patterns

3. Composite

- **Classes:** TaskComponent (abstract base), TaskComposite, HumanTask, AutomatedTask

- **Role:** Treats individual tasks (HumanTask, AutomatedTask) and parallel task groupings

(TaskComposite) uniformly through a common interface.

- **Justification:** Essential for parallel subtask execution, allowing top-level composite rows to manage multiple child steps transparently as a single unit.

4. Decorator

- **Classes:** Decorator (abstract), PriorityDecorator, DueSoonDecorator, ApprovalPendingDecorator

- **Role:** Dynamically attaches supplementary view-only labels (such as "priority", "overdue", "due soon", or "approval pending") to tasks when drawn by the UI without modifying underlying task objects.

- **Justification:** Keeps business logic clean by separating transient UI display states (deadlines, priority badges, locks) from underlying domain models.

5. Facade

- **Classes:** WorkflowEngine

- **Role:** Provides a unified, simplified interface to complex underlying subsystems (including the runner, approval service, participant registry, snapshot history, and workflow definitions).

- **Justification:** Shields the UI from internal subsystem complexity, routing all orchestration requests through a single entry point.

Behavioural Patterns

6. State

- **Classes:** TaskStates (abstract), PendingState, InProgressState, CompletedState, CancelledState

- **Role:** Encapsulates state-specific behavior for human tasks. As operations (start, complete, cancel) are applied, the HumanTask delegates behavior to its current state object, which handles permitted transitions or throws errors for illegal actions.

- **Justification:** Prevents messy conditional logic by cleanly enforcing valid lifecycle transitions (e.g., blocking changes once a task is completed or cancelled).

7. Strategy

- **Classes:** AssignmentRule (abstract), LeastLoadedAssignment, RoleBasedAssignment

- **Role:** Encapsulates family algorithms for assigning tasks to participants. LeastLoadedAssignment picks the person with the fewest open tasks at a given level, while RoleBasedAssignment coordinates finding participants across escalating authority levels.

- **Justification:** Enables pluggable workload distribution policies and hierarchical role escalation without altering core runner logic.

8. Observer

- **Classes:** HumanTask (subject), TaskObserver (abstract), CompletionObserver, AuditObserver

- **Role:** Establishes a one-to-many dependency where changes to a HumanTask's status automatically notify registered observers (such as CompletionObserver to advance the workflow and AuditObserver to record history entries).

- **Justification:** Decouples audit logging and workflow automation from the task model itself, ensuring every status change is automatically captured and acted upon.

9. Iterator

- **Classes:** Iterator (abstract), PendingTaskIterator, DueDateIterator

- **Role:** Provides sequential access to collections of tasks without exposing their underlying representation. PendingTaskIterator filters pending tasks assigned to a specific participant across instances, while DueDateIterator sorts and walks tasks by priority and deadlines.

- **Justification:** Provides clean, dedicated traversal mechanisms for dashboards and sorted deadline views without exposing underlying vector structures.

10. Chain of Responsibility

- **Classes:** RequestHandler (abstract), RoleTierHandler, FallbackHandler

- **Role:** Passes approval requests sequentially up an authority chain of tier handlers (RoleTierHandler). If no authorized person accepts the request by the top tier, the request hits the FallbackHandler to be rejected.

- **Justification:** Allows approval requests to escalate dynamically level-by-level through authority tiers until accepted or safely intercepted by the fallback handler.

11. Memento

- **Classes:** WorkflowSnapshot, SnapshotHistory

- **Role:** Captures and externalizes the internal state of all instances and tasks into a serialized text block, allowing the application to save system states and restore them later via SnapshotHistory.

- **Justification:** Supports application state snapshots and rollbacks (save/restore) for testing, recovery, and time-travel inspection.

Design Decision History

1. Evolution of Task Lifecycle and State Management

- Early Design (hol.pdf / v2): Task status was managed using basic string fields or conditional checks (if/else) directly inside task classes, which created tight coupling and risked illegal status transitions (e.g., attempting to complete a cancelled task).
- Final Design (TIVIDY-UPDATED-UML (1).pdf): Implemented the State Pattern by introducing an abstract TaskStates base class and concrete state classes (PendingState, InProgressState, CompletedState, CancelledState). Each HumanTask now delegates lifecycle operations (start, complete, cancel) directly to its current state object, ensuring invalid operations safely throw exceptions (std::logic_error).

2. Evolution of the Approval and Escalation Mechanism

- Early Design (hol.pdf / v2): Approvals used hardcoded tier classes (LowerRequestHandler, MiddleRequestHandler, HigherRequestHandler), making it rigid and difficult to dynamically scale approval tiers per process definition.
- Final Design (TIVIDY-UPDATED-UML (1).pdf): Evolved into a fully dynamic Chain of Responsibility using generalized RoleTierHandler nodes linked via RequestHandler and terminated by a

FallbackRequestHandler. This allows approval requests to cascade tier-by-tier based on role authority limits, while the ApprovalPendingDecorator locks the task until resolution.

3. Evolution of Observers and Event-Driven Automation

- Early Design (hol.pdf / v2): Tasks and workflow runners were tightly coupled; tasks often had to directly invoke runner methods or log events manually, cluttering the domain logic.
- Final Design (TIVIDY-UPDATED-UML (1).pdf): Formalized the Observer Pattern with TaskObserver and specialized concrete observers (CompletionObserver to trigger workflow advancement, and AuditObserver to record history via AuditEvent). Status changes now cleanly broadcast events without the task needing to know about internal runner mechanisms.

4. Evolution of Task Structuring and Composites

- Early Design (hol.pdf / v2): Handled task groupings with mixed responsibilities, where hierarchy traversal and child management were less defined.
- Final Design (TIVIDY-UPDATED-UML (1).pdf): Refined the Composite Pattern via TaskComponent as the common interface for HumanTask, AutomatedTask, and TaskComposite. This enables the engine to treat single tasks and parallel subtask groupings uniformly

(e.g., requiring all child subtasks in a manufacturing stage to finish before closing the parent composite).

5. Evolution of Assignment Strategies and Builders

- Early Design (hol.pdf / v2): Assignment logic was mixed into actor allocation code, and task construction relied on bloated constructors.
- Final Design (TIVIDY-UPDATED-UML (1).pdf): Separated allocation algorithms into pluggable Strategy classes (LeastLoadedAssignment and RoleBasedAssignment) interfacing with the ParticipantRegistry. Simultaneously, task construction was streamlined using the Builder Pattern (TaskBuilder, HumanTaskBuilder, and ProcessDirector) to assemble complex task hierarchies step-by-step from workflow blueprints.

Implementation Explanation

The system is a generic workflow engine written in C++ in which a workflow is described as data and a runner interprets it. A WorkflowDefinition holds an initial status and a set of TransitionRule entries, each of which maps a status, an action and a flag to the next status and the tasks to create on entering it. Because the insurance claim and manufacturing order processes are both expressed this way, neither requires any change to the engine code. Only the definitions differ, which is how the design supports two very different industries.

Clients interact with the engine through WorkflowEngine, which acts as a Facade. Calls such as startInstance, completeTask and requestApproval check that the instance, task and participant exist, then delegate to the WorkflowRunner, ParticipantRegistry and ApprovalService. This keeps the subsystems out of client code and gives one place where invalid identifiers are rejected. When an instance starts, it is created at the definition's initial status. The runner looks up the matching transition rule and creates the required tasks through the Factory Method classes HumanTaskCreator and AutomatedTaskCreator, using HumanTaskBuilder for human tasks that have many optional fields such as due date, assignee and required role. A Strategy object, either LeastLoadedAssignment or

RoleBasedAssignment, selects the participant, and it returns a null result safely when no one is eligible.

Tasks are organised as a Composite tree. HumanTask, AutomatedTask and TaskComposite share the TaskComponent interface, so the manufacturing order can spawn a composite containing “Cut Raw Steel” and “Program CNC Machine” and treat it like a single task. The composite reports completion only when all of its subtasks are complete. Each HumanTask delegates its lifecycle to a State object (Pending, InProgress, Completed or Cancelled), which validates the requested operation and returns the next state, or throws std::logic_error if the operation is illegal. Decorators such as PriorityDecorator, DueSoonDecorator and ApprovalPendingDecorator wrap a task to add flags and the approval lock without altering the task classes.

Whenever a task changes status it notifies its observers. AuditObserver records the event in the instance’s audit log, distinguishing task completion from subtask completion, and CompletionObserver tells the runner that a task has finished. The runner then re-evaluates the transition rules, cancels any open alternative tasks, advances the instance status and spawns the next tasks, until no rules remain and the engine logs InstanceFinished. When a task needs approval, ApprovalService passes the request along a Chain of Responsibility built from RoleTierHandler objects, ending in a fallback handler, so the request is accepted by the first tier with enough

authority, and the task stays locked in the meantime. Finally, WorkflowSnapshot and SnapshotHistory implement the Memento pattern, letting an instance’s state be saved before a transition and restored later, while DueDateIterator and PendingTaskIterator let the engine list tasks by due date or pending status without exposing the underlying vector.

Links from important requirements to classes, patterns and tests

The requirement that workflows are data-driven and that both industries run on the same engine is implemented by WorkflowDefinition, TransitionRule, WorkflowRunner and the facade WorkflowEngine. It is best evidenced by two end-to-end tests, Engine_EndToEndClaim and Engine_EndToEndManufacturing, which load different definitions into the same engine and run each to InstanceFinished. Together they show that no engine code changes between industries.

The requirement that tasks follow a controlled lifecycle is implemented by HumanTask and the classes TaskStates, PendingState, InProgressState, CompletedState and CancelledState, using the State pattern. Tests such as State_StartThenComplete and State_CancelFromInProgress confirm the legal transitions, while State_CompleteOnCancelled_Throws and State_CompleteOnPending_Throws confirm that illegal operations raise std::logic_error.

The requirement that tasks can be grouped and that a group completes only when all its parts do is implemented by TaskComponent, TaskComposite, HumanTask and AutomatedTask using the Composite pattern. Composite_StaysOpenUntilAllChildrenDone checks the manufacturing behaviour directly, Composite_AddNull_Throws checks the null guard, and

Composite_DestructorFreesChildren checks cleanup, ideally under valgrind or AddressSanitizer.

The requirement for swappable assignment policies is implemented by AssignmentRule, LeastLoadedAssignment and RoleBasedAssignment using the Strategy pattern. Assign_PicksLeastLoaded and Assign_TieKeepsFirst verify the selection logic, and Assign_NoCandidates_ReturnsNull verifies the empty-pool case. The related creation requirement is covered by the Factory Method and Builder classes, tested through Factory_CreatesHumanTask, Factory_CreatesAutomatedTask, Builder_FluentChain and Builder_ResetClearsState.

The requirement that approvals escalate to a sufficiently senior role while the task is locked is implemented by ApprovalService, RequestHandler, RoleTierHandler, the fallback handler and ApprovalPendingDecorator, using Chain of Responsibility and Decorator together. Approval_EscalatesToLevel3 reproduces the insurance claim where the adjuster's $15,000 estimate exceeds their ceiling, Approval_HandledAtFirstEligibleTier confirms the chain stops at the first eligible tier, Approval_NoTier_FallbackOrThrow covers the end of the chain, and Decorator_ApprovalPendingLocksTask confirms the lock.

The requirements that every change is audited and that completion advances the workflow are implemented by TaskObserver, AuditObserver, CompletionObserver and

WorkflowRunner using the Observer pattern. Audit_LogsStartedCompletedCancelled and Audit_SubtaskVsTaskDone verify the audit entries, Completion_AdvancesStatus verifies the transition, and Completion_CancelsOpenAlternatives verifies that open alternatives are cancelled in the same step. A test for the case where no rule matches should also be added, since the behaviour there is a design decision.

The requirement that instance state can be saved and restored is implemented by WorkflowSnapshot and SnapshotHistory using the Memento pattern. Snapshot_SaveRestoreRoundTrip confirms that restoring returns the task tree and statuses to their saved values, and Snapshot_InvalidIndex_Throws confirms the bounds check. The remaining requirements for task flags, task listing and the single entry point are linked in the same way to the decorators, iterators and WorkflowEngine, with tests such as Decorator_NestedLabels, DueDateIterator_Order, PendingIterator_SkipsClosed and Engine_UnknownInstanceOrTask_Throws.

Build, Run, and Test Instructions

```
## Requirements
- g++ 9 or newer (C++17; the Makefile uses -std=c++17)
- GNU Make
- Valgrind (optional, Linux only, for memory checks)

## Build
git clone https://github.com/senzo-codes/TIVIDY
cd TIVIDY

make
make clean

## Run
./TIVIDY

## Test
make test
Expected: <All tests passed (56 assertions in 22 test cases)>

## Memory check (optional)
make valgrind
Expected: "ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)"
```

Git & Engineering Evidence

The repository contains complete source code, a rigorous unit test suite under tests/, UML architectural exports in docs/, and robust memory management verified clean of leaks under Valgrind.